

6-DAY INSTRUCTOR-LED TECHNICAL TRAINING

Core Python with AI-Integrated Development

A fast-track, hands-on bootcamp for experienced developers: master Python's object model, data structures, and standard library — while working AI coding assistants into every stage of the development lifecycle.

- Prior dev experience required
- Python 3.14 · VS Code
- AI Coding Assistant hands-on labs

```
day_01.py

developer =
Engineer(role="builder")

assistant =
configure_ai_assistant()

for day in course.days:
    day.learn()
    day.build_with(assistant)
    day.ship()
```

6-Day Agenda at a Glance

D1

Day 1

Conceptual Foundations, AI Coding Assistant Setup & Python/String Fundamentals

D2

Day 2

AI Coding Agents, Control Flow, Iterators & Core Data Structures (Lists, Tuples, Sets)

D3

Day 3

Dictionaries, Nested & Extended Data Structures, and File Handling

D4

Day 4

Modules & Packages, OOP, Comprehensions, and Exceptions/Debugging/AI-Assisted Testing

D5

Day 5

Core Libraries, Automation, and AI Tools for Developers (Copilot, Context Engineering & Adoption)

D6

Day 6

Capstone Project + Evaluation

DAY 4

Modules & Packages, OOP, Comprehensions, and Exceptions/Debugging/AI-Assisted Testing

- Modules & Packages
- Comprehensions
- Loggers, Exceptions & Debugging - Debugging & Tracebacks
- OOPs & Classes
- Loggers, Exceptions & Debugging - Logging & Exception Handling
- Creating Test Cases Using an AI Assistant

What You'll Be Able To Do

1

Structure Python code into reusable modules and packages, and reason about import resolution via `sys.path` and `.pyc` caching.

2

Design classes with proper encapsulation, inheritance, method overriding, dunder methods, and dataclasses.

3

Replace verbose loops with list/dict/set comprehensions and generator expressions where they improve clarity, and know when not to.

4

Implement production-grade logging and exception handling instead of print-based debugging.

5

Read and diagnose Python tracebacks confidently, and use `breakpoint()/pdb` for interactive debugging.

6

Use an AI coding assistant to generate, review, and iterate on a PyTest test suite for real code.

Modules & Packages

A module is any .py file whose functions/classes/variables can be imported; a package is a directory of modules with an `__init__.py` that gives it a dotted import path.

THE PROBLEM IT SOLVES

Without modules, all code lives in one file, so nothing is reusable, testable in isolation, or shareable across projects.

WHY IT MATTERS

Modules are the unit of code organization and reuse in every real Python codebase and every installable library on PyPI.

WHERE IT'S USED

Every Python project, from a 20-line script to Django/FastAPI apps, is organized as a tree of modules and packages.

REAL-WORLD ANALOGY

"Like splitting a large Java project into separate .java files/JARs, or a Node project into separate files pulled in with require/import."

Import Mechanics

KEY TERMINOLOGY

- Module - a single .py file
- Package - a directory with `__init__.py`
- Namespace - a module's own attribute dict
- `sys.path` - list of directories Python searches for imports

CREATING & IMPORTING

- Any .py file is automatically a module - no export keyword needed
- `import mymodule` loads and runs the file once, caching it in `sys.modules`
- Module name equals the filename without .py
- Re-importing reuses the cached module instead of re-running it

IMPORT VS FROM...IMPORT

- `import module` - access via `module.name`, keeps its namespace
- `from module import name` - pulls name directly into local scope
- `from module import *` - discouraged, pollutes the namespace
- `import module as alias` - common for long or clashing names

DEVELOPER PERSPECTIVE

- Unlike Java, there's no 'export' keyword - everything at module level is importable
- `sys.path` order matters - first match wins, a common source of shadowing bugs

Syntax

```
syntax.py

# mymodule.py
def greet(name):
    return f"Hello, {name}!"

PI = 3.14159

# main.py
import mymodule
print(mymodule.greet("Dev"))      # module.attribute access

from mymodule import greet, PI
print(greet("Team"))              # direct name access

import mymodule as mm
print(mm.PI)
```

HOW IT READS

- `import mymodule` executes `mymodule.py` once and binds the name `'mymodule'`
- `from mymodule import greet` copies a reference to `greet` into the caller's namespace
- `import ... as` gives a local alias, handy for long package paths
- Module-level code (like `PI = 3.14159`) runs immediately on first import
- Constants and functions are just names living in the module's namespace

Basic Example

```
example.py

# file: greetings.py
def greet(name: str) -> str:
    return f"Hello, {name}!"

VERSION = "1.0"

# file: app.py
import greetings

print(greetings.greet("Priya"))
print("Module version:", greetings.VERSION)
```

WHAT'S HAPPENING

- greetings.py has no special syntax marking it as a module - it just exists on disk
- app.py imports it by filename (without .py) and accesses members via dot notation
- Python compiles greetings.py to bytecode and executes it top-to-bottom on first import

CONSOLE OUTPUT

```
Hello, Priya!
Module version: 1.0
```

Practical Example

```
practical_example.py

# project structure:
#   app/main.py
#   shared/utils.py

import sys
import os

# add the sibling 'shared' directory so Python can find it
sys.path.append(os.path.join(os.path.dirname(__file__), "..", "shared"))

import utils # now resolvable via the appended sys.path entry

utils.log_startup("billing-service")
```

WHAT'S HAPPENING

- `sys.path` is just a list of strings - directories Python searches, in order, for a match
- Appending a path at runtime is common for scripts; prefer real packaging in shipped apps
- On import, Python writes compiled bytecode into a `__pycache__/*.pyc` file next to the source
- The `.pyc` is reused on the next run if the `.py` hasn't changed, skipping recompilation
- Deleting `__pycache__` is always safe - Python regenerates it automatically

CONSOLE OUTPUT

```
billing-service started
```

Import Resolution Flow



Packaging, Distribution & What's Next

CLASS-BASED MODULES (PREVIEW)

- A module can define classes instead of just functions - e.g. `models.py` with class `Order`
- Same import rules apply: `import models; models.Order(...)`
- Full class syntax (`__init__`, `self`, inheritance) is the very next topic

DISTRIBUTING YOUR CODE

- `distutils` - original `stdlib` packaging tool, deprecated and removed in 3.12
- `setuptools` - modern standard, builds via `setup.py` or `pyproject.toml`
- Produces a distributable package (`sdist/wheel`) others can `pip` install

PACKAGE MANAGEMENT WITH PIP

- `pip install <name>` - fetches from PyPI into site-packages
- `pip install -r requirements.txt` - installs a pinned dependency set
- `pip freeze > requirements.txt` - captures the current environment

DEVELOPER PERSPECTIVE

- Think `pip` = `npm`/`Maven`; `PyPI` = `npmjs`/`Maven Central`
- Prefer a virtual environment per project - global installs cause version conflicts

Advanced / Developer Example

```
advanced_example.py

import os
import sys
import re
import datetime
import subprocess

# os - filesystem & OS interaction
config_path = os.path.join("data", "config.json")

# sys - interpreter internals & CLI args
py_major = sys.version_info.major

# re - regular expressions
match = re.search(r"\d{3}-\d{4}", "call 555-0199 now")

# datetime - dates & times
launch_day = datetime.date(2026, 9, 13)
```

WHAT'S HAPPENING

- os: paths, environment variables, filesystem ops (os.path.join, os.getenv, os.listdir)
- sys: interpreter state - sys.argv, sys.path, sys.exit(), sys.version_info
- re: pattern matching/validation - search, match, findall, sub
- datetime: date/time arithmetic and formatting - date objects, timedelta, strftime
- subprocess: launch and capture output from external processes/CLI tools

CONSOLE OUTPUT

```
data/config.json 3 555-0199 2026-09-13 hi
```

Common Mistakes

✗ INCORRECT

```
# a.py
import b
# b.py
import a
```

✗ INCORRECT

```
sys.path.append(path) # scattered everywhere
```

✗ INCORRECT

```
from os import *
```

✗ INCORRECT

```
# hand-editing a .pyc file
```

✗ INCORRECT

```
pip install requests # no venv
```

✓ CORRECT

```
# move shared logic into a third module c.py
# that both a.py and b.py import from
```

✓ CORRECT

```
# use a proper package layout / pyproject.toml
# instead of manual sys.path hacks
```

✓ CORRECT

```
import os
os.path.join(...)
```

✓ CORRECT

```
# edit the .py source; let Python
# regenerate __pycache__ automatically
```

✓ CORRECT

```
python -m venv .venv
source .venv/bin/activate
pip install requests
```

Common Mistakes — Why It Matters

- Mistake 1: Circular top-level imports raise ImportError/AttributeError depending on import order
- Mistake 2: Manual sys.path edits are fragile and break once the project is packaged or moved
- Mistake 3: Star imports make it unclear whether a name is yours or from the module
- Mistake 4: .pyc is a compiled cache, not the source of truth - edits to it are silently overwritten
- Mistake 5: Global installs cause dependency version conflicts across projects

Best Practices

PEP 8 & READABILITY

- One import per line; group stdlib / third-party / local imports separately
- Avoid wildcard imports; import only what you use

MAINTAINABILITY

- Keep modules focused (single responsibility) - split large files by concern
- Use `__init__.py` to define a clean public API for a package

PERFORMANCE

- Avoid heavy work at module import time - it runs on every import
- Let Python manage `__pycache__`; don't hand-edit or commit it

SECURITY

- Pin dependency versions in `requirements.txt` / `pyproject.toml`
- Vet third-party packages before pip install - supply-chain risk

AI-Assisted Development

PROMPT

Review this module's import section for unused imports and star-imports, and suggest a cleaner PEP 8-compliant import ordering plus a minimal pyproject.toml for packaging it.

```
before_ai.py

from utils import *
import os, sys
import requests

def fetch(url):
    return requests.get(url).json()
```

AI ASSISTANT SUGGESTS

- Flags 'from utils import *' as a wildcard-import risk; suggests explicit names
- Splits 'import os, sys' into separate lines per PEP 8
- Groups stdlib (os, sys) above third-party (requests) with a blank line between
- Suggests adding requests to requirements.txt with a version pin
- Proposes a minimal pyproject.toml if this is meant to be an installable package

DEVELOPER MUST STILL VERIFY

- Confirm every suggested unused-import removal is actually unused elsewhere
- Check the pinned version doesn't break compatibility with other dependencies
- Verify pyproject.toml metadata (name, version) before publishing
- Re-run the code after applying suggestions

Hands-on Exercise

PROBLEM STATEMENT

Create a module `string_tools.py` with functions to reverse a string and count vowels. Import it two ways (`import` and `from...import`) into a driver script, then use `pip` to check whether the `'requests'` package is installed.

INPUT

```
text = "Hello Python Training"
```

EXPECTED OUTPUT

```
Reversed: gniniarT nohtyP olleH  
Vowel count: 6
```

HINTS (no full solution here — give it a try first!)

- Use `import string_tools as st` for the aliased form
- `pip show requests` tells you if/where a package is installed
- `vowels = 'aeiouAEIOU'` is the filter set for part 2

OOPs & Classes

Python classes bundle state (attributes) and behavior (methods) into objects, using the same class/instance model as Java, C#, or JS, but with dynamic typing and no separate interface/header syntax.

THE PROBLEM IT SOLVES

Plain functions and dicts don't scale well once you need many related pieces of state and behavior bound together with invariants.

WHY IT MATTERS

Nearly every non-trivial Python codebase - frameworks, ORMs, data models - is built from classes.

WHERE IT'S USED

Django models, custom exceptions, data classes for API payloads, GUI widgets, service objects.

REAL-WORLD ANALOGY

"Same mental model as a Java/C# class or a JS class - a blueprint (class) stamped into concrete objects (instances)."

Defining a Class

KEY TERMINOLOGY

- Class - the blueprint/type definition
- Instance - an object created from a class
- self - the instance itself, explicit first parameter
- Attribute - a variable bound to an instance or class

INSTANCE VS CLASS STATE

- `__init__` is the constructor, called automatically on `ClassName(...)`
- Instance attributes are set via `self.name = value`, usually inside `__init__`
- Class attributes are defined directly in the class body, shared by all instances
- Instance methods take `self` as the first parameter, bound automatically

ENCAPSULATION CONVENTIONS

- No real 'private' keyword - Python relies on naming convention, not enforcement
- `_name` (single underscore) - 'protected', internal use, not enforced by the interpreter
- `__name` (double underscore) - name-mangled to `_ClassName__name`, avoids subclass collisions
- Everything is technically accessible - Python trusts the developer

DEVELOPER PERSPECTIVE

- `self` is explicit here vs Java/C#'s implicit 'this' - always the first parameter you write
- No access-modifier keywords (`public/private/protected`) like Java/C# - just convention

Syntax

```
syntax.py

class Employee:
    company = "Acme Corp"          # class attribute (shared)

    def __init__(self, name, salary):
        self.name = name           # instance attribute
        self._salary = salary      # "protected" by convention

    def give_raise(self, amount):   # instance method
        self._salary += amount
        return self._salary

e = Employee("Dana", 70000)
print(e.name, e.company)
print(e.give_raise(5000))
```

HOW IT READS

- `class Employee:` starts the class body - no braces, indentation defines scope
- `__init__(self, ...)` runs once per object, right after creation
- `self.name` binds an attribute to THIS instance; `company` lives on the class itself
- `e.give_raise(5000)` is sugar for `Employee.give_raise(e, 5000)` - `self` is passed automatically
- `_salary`'s leading underscore signals 'internal', but nothing blocks `e._salary` access

Basic Example

example.py

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_from_origin(self):
        return (self.x ** 2 + self.y ** 2) ** 0.5

p = Point(3, 4)
print(p.distance_from_origin())
```

WHAT'S HAPPENING

- Point() calls __init__ automatically, passing the new object as self
- self.x/self.y are per-instance - each Point has its own coordinates
- distance_from_origin reads self.x/self.y - no need to pass them explicitly

CONSOLE OUTPUT

5.0

Practical Example

```
practical_example.py

class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def describe(self):
        return f"{self.make} {self.model}"

class ElectricCar(Vehicle):
    def __init__(self, make, model, battery_kwh):
        super().__init__(make, model) # reuse parent init
        self.battery_kwh = battery_kwh

    def describe(self): # method override
        base = super().describe() # call parent version
        return f"{base} ({self.battery_kwh} kWh EV)"
```

WHAT'S HAPPENING

- ElectricCar(Vehicle) inherits all of Vehicle's attributes and methods
- super().__init__(...) calls the parent constructor instead of duplicating logic
- describe() is overridden - Python resolves the call based on the instance's actual type
- super().describe() inside the override still calls the parent's version explicitly
- Same polymorphism you'd expect from Java/C# - just no 'virtual'/'override' keyword needed

CONSOLE OUTPUT

```
Toyota Corolla
Tesla Model 3 (75kWh EV)
```

Python OOP vs Java/C#/JavaScript

Feature	Python	Java / C#	JavaScript
Instance ref	<code>self</code> (explicit param)	<code>this</code> (implicit)	<code>this</code> (implicit, context-dependent)
Constructor	<code>__init__</code>	<code>ClassName()</code> / Constructor	<code>constructor()</code>
Access control	Convention only (<code>_x</code> , <code>__x</code>)	<code>private</code> / <code>protected</code> / <code>public</code> keywords	<code>#</code> for true private fields
Method override	Just <code>redefine</code> + <code>super()</code>	<code>@Override</code> + <code>super/base</code>	Just <code>redefine</code> + <code>super()</code>

Advanced / Developer Example

```
advanced_example.py

from dataclasses import dataclass

class Money:
    def __init__(self, amount):
        self.amount = amount

    def __repr__(self):
        # unambiguous, dev-facing
        return f"Money({self.amount})"

    def __str__(self):
        # readable, user-facing
        return f"${self.amount:.2f}"

    def __eq__(self, other):
        return isinstance(other, Money) and self.amount == other.amount

@dataclass
class Point3D:
    # __init__ / __repr__ / __eq__ auto-generated
    x: float
```

WHAT'S HAPPENING

- `__repr__` is for developers/debugging; `__str__` is for `print()`/end users
- `__eq__` overrides `==` so two `Money` objects compare by value, not identity
- `@dataclass` auto-generates `__init__`, `__repr__`, and `__eq__` from type-annotated fields
- Use dataclasses for simple data-holder classes to cut boilerplate dramatically
- Without `__eq__`, `==` falls back to identity comparison, like default Java `Object.equals()`

CONSOLE OUTPUT

```
$10.00 True Money(10)
Point3D(x=1, y=2, z=3)
```

Common Mistakes

✗ INCORRECT

```
def __init__(new_instance, x):  
    new_instance.x = x
```

✓ CORRECT

```
def __init__(self, x):  
    self.x = x
```

✗ INCORRECT

```
class Counter:  
    count = 0  
    def inc(self):  
        count += 1
```

✓ CORRECT

```
class Counter:  
    def __init__(self):  
        self.count = 0  
    def inc(self):  
        self.count += 1
```

✗ INCORRECT

```
if a == b: # relies on default equality
```

✓ CORRECT

```
def __eq__(self, other):  
    return self.value == other.value
```

✗ INCORRECT

```
class Child(Base):  
    def __init__(self):  
        self.x = 1 # never calls super()
```

✓ CORRECT

```
class Child(Base):  
    def __init__(self):  
        super().__init__()  
        self.x = 1
```

Common Mistakes — Why It Matters

- Mistake 1: `self` is only a convention but virtually universal - anything else confuses every other Python developer
- Mistake 2: Forgetting `self`. treats `count` as a local/class variable, not the instance attribute you meant
- Mistake 3: Without `__eq__`, `==` compares object identity, not field values, unlike Java's overridable `equals()`
- Mistake 4: Skipping `super().__init__()` silently drops the parent's setup logic

Best Practices

PEP 8 & READABILITY

- Class names in CapWords (PascalCase); methods/attributes in snake_case
- One blank line between methods, two before a class definition

MAINTAINABILITY

- Prefer composition over deep inheritance chains
- Use @dataclass for plain data containers instead of hand-written boilerplate

PERFORMANCE

- __slots__ can reduce memory for classes with millions of instances

AI-Assisted Development

PROMPT

Review this class for missing encapsulation or validation, a missing `__eq__`/`__repr__`, and suggest whether it should be a `@dataclass`.

before_ai.py

```
class Item:
    def __init__(self, sku, price):
        self.sku = sku
        self.price = price
```

AI ASSISTANT SUGGESTS

- Notes the class has no custom `__eq__`/`__repr__` - comparisons use identity, prints are unhelpful
- Suggests `@dataclass` since it's a plain data container with no custom methods
- Flags 'price' as publicly mutable with no validation (e.g. negative prices allowed)
- Recommends a validator or property if price must stay non-negative
- Suggests type hints (`sku: str`, `price: float`) for clarity

DEVELOPER MUST STILL VERIFY

- Confirm `@dataclass` doesn't remove behavior you actually need (custom methods still allowed)
- Test `__eq__`/`__repr__` output manually after conversion
- Check nothing else in the codebase relied on identity comparison for this class

Hands-on Exercise

PROBLEM STATEMENT

Model a Shape base class with an `area()` method, then Circle and Rectangle subclasses that override it. Add `__str__` so printing an instance shows its name and area rounded to 2 decimals.

INPUT

```
shapes = [Circle(3), Rectangle(4, 5)]
```

EXPECTED OUTPUT

```
Circle: area=28.27  
Rectangle: area=20.00
```

HINTS (no full solution here — give it a try first!)

- Use `super().__init__()` if Shape stores a shared name attribute
- `math.pi` is available from the standard math module
- Format with `f"{area:.2f}"` inside `__str__`

Comprehensions

A comprehension builds a new list/dict/set (or a lazy generator) from an iterable in a single expression, replacing a for-loop plus `.append()` pattern.

THE PROBLEM IT SOLVES

Manually building collections with an empty container, a for-loop, and `.append()/[k]=` is verbose and easy to get wrong.

WHY IT MATTERS

Comprehensions are idiomatic Python - reviewers and AI assistants both expect them for simple transform/filter logic.

WHERE IT'S USED

Data transformation, filtering API/DB results, building lookup dicts, deduplication, memory-efficient pipelines.

REAL-WORLD ANALOGY

"Similar in spirit to Java Streams (`.map().filter().collect()`) or JS array `.map().filter()`, expressed as one declarative expression."

List, Dict & Set Comprehensions

KEY TERMINOLOGY

- Comprehension - expression + for + optional if, producing a new collection
- Generator expression - lazy version using () instead of []
- Iterable - anything a comprehension loops over (list, range, file, etc.)

LIST COMPREHENSION

- [expr for item in iterable] builds a full list
- Add 'if condition' to filter items
- Multiple 'for' clauses nest, evaluated left to right

DICT / SET COMPREHENSION

- {k: v for ...} builds a dict; {expr for ...} builds a set
- The dict form needs a key:value pair per iteration
- The set form auto-deduplicates results, like a normal set

DEVELOPER PERSPECTIVE

- Comprehensions are expressions - they can be passed directly as function arguments
- If it needs more than one condition/nesting level to read, use a plain loop instead

Syntax

```
syntax.py

squares = [n * n for n in range(6)]           #
list
evens_sq = [n * n for n in range(6) if n % 2 == 0] #
list + filter
lookup = {n: n * n for n in range(4)}         #
dict
unique_lens = {len(w) for w in ["a", "bb", "cc"]} #
set
lazy = (n * n for n in range(6))              #
generator expression
```

HOW IT READS

- Square brackets [] build a list; curly braces {} build a dict (k:v) or a set (single expr)
- Parentheses () build a generator expression, evaluated lazily, one item at a time
- 'if condition' after the for-clause filters which items produce output
- Multiple 'for' clauses are allowed for nested loops, evaluated left to right
- The loop variable (n, w) is scoped to the comprehension in Python 3, not leaked outward

Basic Example

example.py

```
names = ["alice", "Bob", "CHARLIE"]
titled = [name.title() for name in names]
print(titled)
```

WHAT'S HAPPENING

- [expr for item in iterable] applies name.title() to every element
- The result is a brand-new list; 'names' itself is left untouched

CONSOLE OUTPUT

```
['Alice', 'Bob', 'Charlie']
```

Practical Example

```
practical_example.py

records = [
    {"id": 1, "status": "active", "region": "US"},
    {"id": 2, "status": "inactive", "region": "EU"},
    {"id": 3, "status": "active", "region": "EU"},
]

active_by_id = {r["id"]: r for r in records if r["status"] == "active"}
regions = {r["region"] for r in records}

print(sorted(active_by_id.keys()))
print(sorted(regions))
```

WHAT'S HAPPENING

- The dict comprehension turns a list of records into an id -> record lookup in one line
- Only records passing the 'if' filter are included
- The set comprehension collects distinct regions, automatically removing duplicates

CONSOLE OUTPUT

```
[1, 3]
['EU', 'US']
```

Advanced / Developer Example

```
advanced_example.py

# generator expression - lazy, one-shot, no 'yield' needed
squares_gen = (n * n for n in range(1_000_000))

# generator FUNCTION - same laziness, but with multi-step logic and 'yield'
def even_squares(limit):
    for n in range(limit):
        if n % 2 == 0:
            yield n * n

print(next(squares_gen))           # pulls one value, doesn't build the list
print(list(even_squares(6)))
```

WHAT'S HAPPENING

- `squares_gen` computes nothing until iterated - no 1,000,000-item list ever built in memory
- `next()` pulls exactly one value on demand from the generator expression
- `even_squares` uses 'yield' because the logic needs a statement (if), not just an expression
- Both are generators (support `next()`); comprehension syntax is just a shortcut for simple cases
- Prefer generator expressions for simple pipelines, generator functions for complex logic

CONSOLE OUTPUT

```
0
[0, 4, 16]
```

Comprehensions vs map/filter/lambda

Approach	Code	Advantages	Disadvantages
Comprehension	<pre>[x*2 for x in nums if x > 0]</pre>	Readable, Pythonic, one expression	Can get dense if over-nested
map/filter/lambda	<pre>list(map(lambda x: x*2, filter(lambda x: x>0, nums)))</pre>	Familiar to functional/JS devs	Nested calls read right-to-left
Plain for-loop	<pre>result = [] for x in nums: if x>0: result.append(x*2)</pre>	Clear for complex, multi-step logic	Verbose for simple transforms

Common Mistakes

✗ INCORRECT

```
[x for x in data if x>0 if x<100
 for y in x.items() if y]
```

✗ INCORRECT

```
[print(x) for x in items]
```

✗ INCORRECT

```
big = [x*x for x in range(100_000_000)]
```

✗ INCORRECT

```
squares = [n*n for n in range(5)]
print(n)  # expects leftover n
```

✓ CORRECT

```
# break into a named function
# or a plain for-loop instead
```

✓ CORRECT

```
for x in items:
    print(x)
```

✓ CORRECT

```
big = (x*x for x in range(100_000_000))
```

✓ CORRECT

```
for n in range(5):
    pass
print(n)  # only a plain loop leaks n
```

Common Mistakes — Why It Matters

- Mistake 1: More than one for/if clause quickly becomes unreadable - keep comprehensions to a one-liner
- Mistake 2: Comprehensions build a new collection; using one only for side effects wastes memory on a throwaway list
- Mistake 3: A list comprehension over a huge range materializes everything in memory; a generator stays lazy
- Mistake 4: Comprehension loop variables are scoped locally in Python 3 and never leak outward

Best Practices

PEP 8 & READABILITY

- Keep comprehensions to one logical line; wrap only if still scannable
- Prefer a descriptive variable name over single letters for anything non-trivial

MAINTAINABILITY

- If it needs a comment to explain it, refactor to a loop or a named function

PERFORMANCE

- Use generator expressions instead of list comprehensions when iterating only once
- `sum(x for x in ...)` avoids building an intermediate list vs `sum([x for x in ...])`

AI-Assisted Development

PROMPT

Simplify this loop into an idiomatic comprehension where it improves readability, but keep it as a loop if a comprehension would be overly dense.

before_ai.py

```
result = []
for order in orders:
    if order['status'] == 'paid':
        result.append(order['total'])
```

AI ASSISTANT SUGGESTS

- Suggests: `[o['total'] for o in orders if o['status'] == 'paid']`
- Confirms this stays readable as a comprehension (single filter, single expression)
- Recommends NOT converting nested/multi-condition loops the same way
- Suggests a generator expression instead if 'result' is only summed/iterated once
- Points out `sum(o['total'] for o in orders if o['status']=='paid')` skips the intermediate list

DEVELOPER MUST STILL VERIFY

- Confirm the comprehension produces identical output to the original loop on real data
- Check readability with a teammate - don't accept it if it's actually harder to read
- Verify no side effects (like logging) were silently dropped from the original loop

Hands-on Exercise

PROBLEM STATEMENT

Given a list of word strings, build: (1) a list of uppercase words longer than 3 characters, (2) a dict mapping each word to its length, (3) a list of only the palindromes using a generator expression.

INPUT

```
words = ['python', 'level', 'hi', 'radar', 'ok']
```

EXPECTED OUTPUT

```
['PYTHON', 'LEVEL', 'RADAR']  
{ 'python': 6, 'level': 5, 'hi': 2, 'radar': 5, 'ok': 2 }  
['level', 'radar']
```

HINTS (no full solution here — give it a try first!)

- A palindrome equals its own reverse: `w == w[::-1]`
- Wrap the generator in `list(...)` only to print it for verification
- `len(w) > 3` is the filter condition for part 1

Logging & Exception Handling

Logging records what a running program is doing to a configurable destination; exception handling lets code react to and recover from runtime errors instead of crashing.

THE PROBLEM IT SOLVES

print()-based status messages and unhandled crashes don't scale to production services that must stay observable and resilient.

WHY IT MATTERS

Every production Python service relies on structured logging and deliberate exception handling for reliability and diagnosis.

WHERE IT'S USED

Web APIs, batch jobs, CLI tools - anywhere failures must be caught, logged, and sometimes recovered from.

REAL-WORLD ANALOGY

"Like a flight data recorder (logging) plus seatbelts and airbags (exception handling) - one records what happened, the other contains the damage."

Logging in Python

KEY TERMINOLOGY

- Logger - named object you log through, `logging.getLogger(__name__)`
- Level - severity of a message: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Handler - destination for log records (console, file, network)
- Formatter - controls the text layout of each log line

HOW IT WORKS

- `logging.basicConfig()` sets a quick default logger/handler/format for the whole app
- `logger.info(...)/logger.error(...)` etc. emit at that severity
- Only messages at or above the configured level are actually output
- Handlers can fan the same logger out to multiple destinations (console + file)

IMPORTANT RULES

- Never use `print()` for operational/status messages in production code
- Use `logger = logging.getLogger(__name__)` per module, not one shared global logger

DEVELOPER PERSPECTIVE

- `print()` can't be filtered, redirected, or leveled at runtime - logging can, with no code changes
- Use `logger.exception()` inside an except block to auto-capture the traceback

Syntax

```
syntax.py

import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s"
)
logger = logging.getLogger(__name__)

logger.debug("won't show - below INFO level")
logger.info("service started")
logger.warning("disk usage at 85 percent")
logger.error("failed to connect to database")
```

HOW IT READS

- `basicConfig()` runs once, typically at app startup, wiring the root logger
- `level=logging.INFO` means DEBUG messages are suppressed everywhere by default
- `%(asctime)s / %(levelname)s / %(name)s / %(message)s` are standard format placeholders
- `getLogger(__name__)` names the logger after the current module, useful for tracing
- Each call (`debug/info/warning/error/critical`) maps directly to a severity level

Basic Example

```
example.py

import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("orders")

def process_order(order_id, amount):
    logger.info("processing order %s for $%.2f", order_id, amount)
    return amount * 1.08

process_order(101, 49.99)
```

WHAT'S HAPPENING

- Passing args as `logger.info(fmt, a, b)` defers string formatting until actually logged
- This avoids the cost of building the string when the level would filter it out anyway

CONSOLE OUTPUT

```
INFO:orders:processing order 101 for $49.99
```

Exception Handling

KEY TERMINOLOGY

- Exception - an object raised when an error occurs during execution
- try/except - a block that attempts code and reacts to specific error types
- raise - explicitly signals an error, built-in or custom
- Custom exception - a class inheriting from Exception (or a subclass)

CONTROL FLOW

- try runs first; except only runs if a matching exception is raised
- else runs only if the try block succeeded with NO exception
- finally always runs - on success, on failure, or even around a return

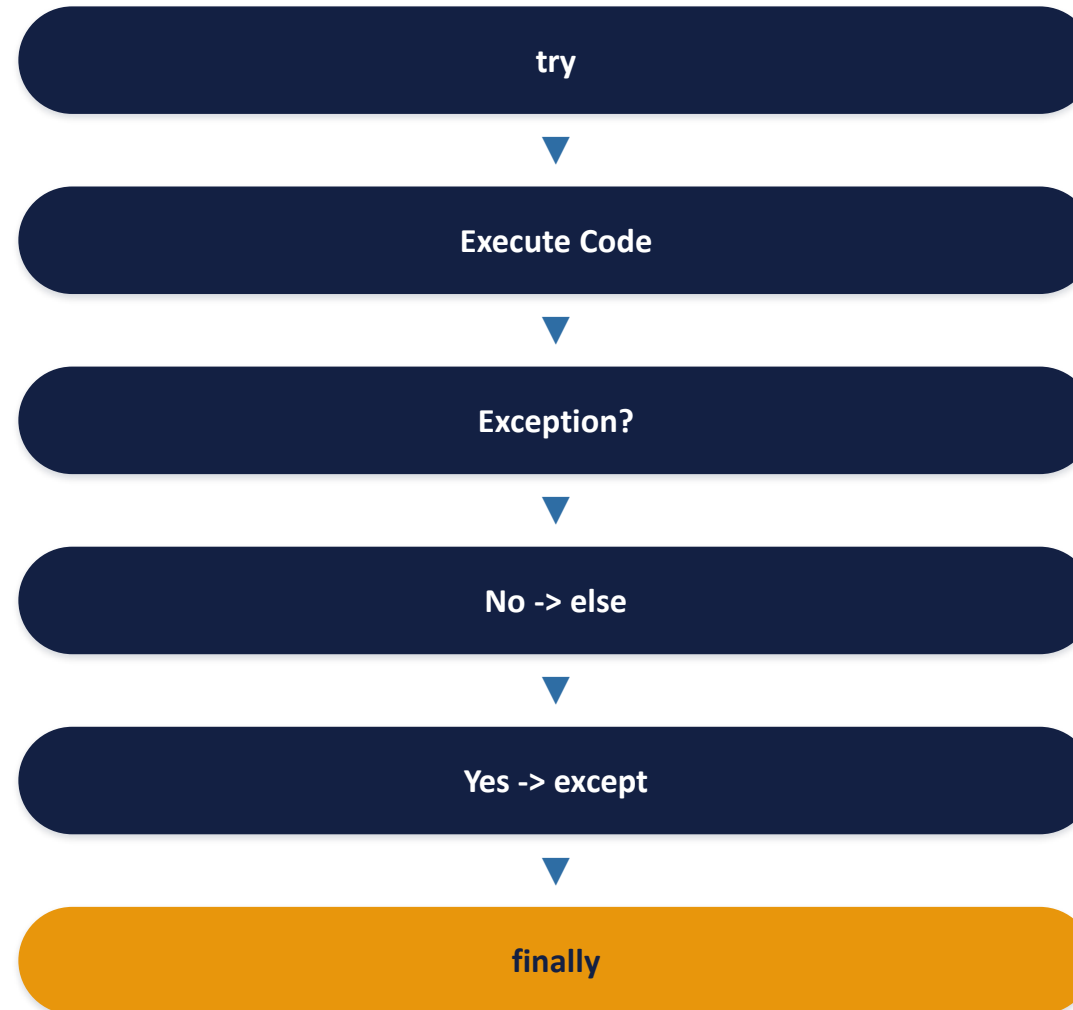
IMPORTANT RULES

- Catch the most specific exception type you can, never a bare except:
- Custom exceptions should subclass Exception, not BaseException directly

DEVELOPER PERSPECTIVE

- else is Python-specific and often skipped by developers from Java/C#/JS - use it to separate happy-path code from cleanup

Exception Handling Control Flow



Practical Example

```
practical_example.py

class InsufficientStockError(Exception):
    """Raised when an order exceeds available inventory."""
    pass

def ship_order(item, qty, stock):
    try:
        if qty > stock[item]:
            raise InsufficientStockError(
                f"only {stock[item]} units of {item} left, requested {qty}"
            )
        stock[item] -= qty
    except InsufficientStockError as e:
        print(f"Order rejected: {e}")
    else:
        print(f"Shipped {qty} x {item}")
    finally:
        print(f"Stock check complete for {item}")
```

WHAT'S HAPPENING

- `InsufficientStockError(Exception)` defines a domain-specific error type, not a generic one
- `raise` constructs and throws it with a descriptive message, caught by the matching `except`
- `else` only prints 'Shipped' when no exception occurred in `try`
- `finally` runs in BOTH calls regardless of success or failure - ideal for cleanup/logging

CONSOLE OUTPUT

```
Order rejected: only 5 units of widget left,
requested 10
Stock check complete for widget
Shipped 3 x widget
Stock check complete for widget
```

Common Mistakes

✗ INCORRECT

```
except:  
    pass
```

✓ CORRECT

```
except ValueError as e:  
    logger.exception(e)
```

✗ INCORRECT

```
print('error:', e)
```

✓ CORRECT

```
logger.error('failed: %s', e)
```

✗ INCORRECT

```
try:  
    risky()  
except Exception:  
    risky() # retry blindly forever
```

✓ CORRECT

```
# retry with a max attempt count and backoff
```

✗ INCORRECT

```
class MyError(BaseException):  
    pass
```

✓ CORRECT

```
class MyError(Exception):  
    pass
```

Common Mistakes — Why It Matters

- Mistake 1: A bare except: silently swallows everything, including KeyboardInterrupt and real bugs
- Mistake 2: print output can't be filtered, redirected, or disabled per environment; logging can
- Mistake 3: Unbounded blind retries can hang or hammer a failing dependency
- Mistake 4: BaseException also covers SystemExit/KeyboardInterrupt - custom errors should subclass Exception

Best Practices

PEP 8 & READABILITY

- Name custom exceptions with an Error suffix (InsufficientStockError)
- Keep try blocks small - wrap only the line(s) that can actually fail

MAINTAINABILITY

- Define a small hierarchy of app-specific exceptions (AppError -> ValidationError, NotFoundError)
- Use `logger.exception()` inside except to auto-capture the traceback

PERFORMANCE

- Exceptions are for exceptional cases, not normal control flow

SECURITY

- Never leak internal exception details (stack traces, SQL, paths) directly to end users

AI-Assisted Development

PROMPT

Review this exception handling for overly broad except clauses, missing logging, and swallowed errors; suggest a custom exception hierarchy.

```
before_ai.py

def load_config(path):
    try:
        with open(path) as f:
            return f.read()
    except:
        return None
```

AI ASSISTANT SUGGESTS

- Flags the bare 'except:' as catching everything, including unrelated bugs
- Suggests catching FileNotFoundError specifically instead of all exceptions
- Recommends logging the error (logger.exception) instead of silently returning None
- Proposes a custom ConfigLoadError to distinguish this failure from other None returns
- Notes the caller currently can't tell 'file missing' apart from 'file empty'

DEVELOPER MUST STILL VERIFY

- Confirm the narrowed except still covers every realistic failure mode
- Check that logging doesn't leak sensitive file paths/content in production logs
- Re-test both the success and failure paths after the change

Hands-on Exercise

PROBLEM STATEMENT

Write `transfer_funds(from_acct, to_acct, amount)` that raises a custom `InsufficientFundsError` when the source account lacks funds, logs an `INFO` message on success and an `ERROR` message on failure (no print), and always logs a finally message noting the attempt completed.

INPUT

```
accounts = {'A': 100, 'B': 50}
transfer_funds('A', 'B', 30)
transfer_funds('B', 'A', 999)
```

EXPECTED OUTPUT

```
INFO: transferred 30 from A to B
INFO: transfer attempt complete
ERROR: insufficient funds in B for transfer of 999
INFO: transfer attempt complete
```

HINTS (no full solution here — give it a try first!)

- Raise a custom `InsufficientFundsError(Exception)` subclass
- Put the 'attempt complete' log line in finally
- Use `logger.error(str(e))` inside the except block

Debugging & Tracebacks

Debugging is the process of locating why code doesn't behave as expected, using error messages, tracebacks, and interactive tools instead of guesswork.

THE PROBLEM IT SOLVES

Scattering `print()` statements is slow and often misses the real fault, especially in deeper call stacks.

WHY IT MATTERS

Reading a traceback correctly is the fastest way to find a bug's true location - a core daily skill for any Python developer.

WHERE IT'S USED

Every debugging session, from a local script crash to a production incident triaged from log output.

REAL-WORLD ANALOGY

"A traceback is like a GPS breadcrumb trail showing exactly which route (call stack) led to the crash site."

Reading Errors & Tracebacks

KEY TERMINOLOGY

- Traceback - the call-stack history printed when an unhandled exception propagates
- Exception type - the last line of a traceback (e.g. ValueError: ...)
- Stack frame - one function call's context shown in the traceback
- breakpoint() - built-in that drops into the debugger at that exact line

READING A TRACEBACK

- Read tracebacks BOTTOM-UP - the last line is the actual error type and message
- Each 'File ..., line ..., in function' block above it is one call frame, outermost first
- The source line shown is exactly where execution was when it failed

PRINT() DEBUGGING & ITS LIMITS

- print(x) is fast for a quick check but must be manually added and removed everywhere
- Doesn't preserve state or allow interactive inspection - only shows what you explicitly printed

DEVELOPER PERSPECTIVE

- Read the exception type + message first, then walk the traceback to the frame you actually own

Basic Example

```
example.py

def parse_age(text):
    return int(text)

def register_user(name, age_text):
    age = parse_age(age_text)
    return f"{name} is {age}"

print(register_user("Sam", "twenty"))
```

WHAT'S HAPPENING

- Calling this raises ValueError: invalid literal for int() with base 10: 'twenty'
- Bottom-up: the error is a ValueError from int(), inside parse_age, called from register_user
- The traceback names both frames (register_user, parse_age) plus the exact failing line

CONSOLE OUTPUT

```
Traceback (most recent call last):
  File "app.py", line 8, in <module>
    print(register_user("Sam", "twenty"))
  File "app.py", line 5, in register_user
    age = parse_age(age_text)
  File "app.py", line 2, in parse_age
    return int(text)
ValueError: invalid literal for int() with
base 10: 'twenty'
```

Debugging Principles & Tools

DEBUGGING PRINCIPLES

- Reproduce reliably before fixing - a bug you can't reproduce, you can't verify fixed
- Bisect: isolate halves of the code to narrow down the failing region
- Check assumptions against real values, don't assume input matches expectations

TRICKS

- Read the exception type first - it usually narrows the search immediately
- Reduce to the smallest failing example before debugging the full system

USING A DEBUGGER

- `breakpoint()` (3.7+) pauses execution and opens pdb at that exact line
- pdb commands: n (next line), s (step into), c (continue), p expr (print), l (list source)
- IDEs (VS Code, PyCharm) wrap pdb with clickable breakpoints and variable inspectors

DEVELOPER PERSPECTIVE

- A debugger lets you inspect live state instead of guessing what to print next

Syntax

```
syntax.py

def compute_discount(price, pct):
    breakpoint()          # pauses here, opens (Pdb) prompt
    discount = price * pct / 100
    return price - discount

compute_discount(200, 15)

# at the (Pdb) prompt you could type:
# p price      -> 200
# p pct        -> 15
# n            -> execute next line
# c            -> continue running
```

HOW IT READS

- `breakpoint()` is a zero-import shortcut for `import pdb; pdb.set_trace()`
- Execution pauses exactly at that line, before it runs
- `p <expr>` prints any expression's current value in that frame
- `n` steps to the next line in the current frame; `s` steps INTO a function call
- `c` resumes normal execution until the next breakpoint or program end

Practical Example

practical_example.py

```
def total_price(items):
    total = 0
    for item in items:
        total += item["price"] * item["qty"]
    return total

cart = [{"price": 10, "qty": 2}, {"price": "5", "qty": 3}] # bug: price is a string
print(total_price(cart))
```

WHAT'S HAPPENING

- This raises `TypeError: unsupported operand type(s) for +=: 'int' and 'str'` on the second item
- Dropping `breakpoint()` before the loop lets you 'p item' each iteration to spot the bad string price
- Without a debugger you'd need to add `print(item)` manually and re-run to see the same thing
- The fix is validating/casting price to a number where the cart item is built, not where it's used

CONSOLE OUTPUT

```
Traceback (most recent call last):
  File "cart.py", line 8, in <module>
    print(total_price(cart))
  File "cart.py", line 4, in total_price
    total += item["price"] * item["qty"]
TypeError: unsupported operand type(s) for
+=: 'int' and 'str'
```

Best Practices

PEP 8 & READABILITY

- Remove `breakpoint()/pdb.set_trace()` calls before committing code

MAINTAINABILITY

- Log enough context (inputs, IDs) at ERROR level so tracebacks are actionable without a debugger

PERFORMANCE

- Don't leave verbose DEBUG-level logging enabled on hot production paths

AI-Assisted Development

PROMPT

Here is a traceback and the surrounding function. Explain the root cause in plain language and suggest a minimal fix.

```
before_ai.py

def average(nums):
    return sum(nums) / len(nums)

print(average([]))
# Traceback: ZeroDivisionError: division by
zero
```

AI ASSISTANT SUGGESTS

- Explains `len(nums)` is 0 for an empty list, so the division fails
- Points to the exact line (`return sum(nums)/len(nums)`) as the fault location
- Suggests guarding with `'if not nums: raise ValueError("nums must not be empty")'`
- Notes the caller should decide the right empty-input behavior, not just the function
- Suggests adding a unit test for the empty-list case

DEVELOPER MUST STILL VERIFY

- Confirm the suggested fix matches actual business rules for empty input
- Re-run with both empty and non-empty input after applying the fix
- Check no other caller depended on the old (crashing) behavior

Hands-on Exercise

PROBLEM STATEMENT

Given a function with a hidden off-by-one bug in an index expression, use `breakpoint()/pdb` (or your IDE's debugger) to step through, inspect `len(items)`, and identify exactly where the index goes out of range; then read and interpret the resulting traceback.

INPUT

```
def last_two(items):  
    return [items[len(items)-1], items[len(items)]] # bug
```

EXPECTED OUTPUT

```
IndexError: list index out of range, pinpointed to the second  
index expression
```

HINTS (no full solution here — give it a try first!)

- Step through with `n` and inspect `len(items)` with `p`
- The valid index range is 0 to `len(items)-1`
- Fix by using `items[-1]` and `items[-2]` instead

Creating Test Cases Using an AI Assistant

Using an AI coding assistant to draft PyTest test modules for existing code, then reviewing and iterating on them like any other AI-generated code.

THE PROBLEM IT SOLVES

Hand-writing a thorough test suite covering both happy paths and edge cases is time-consuming and easy to under-cover manually.

WHY IT MATTERS

AI assistants are effective at drafting boilerplate-heavy test code fast - but only if the developer reviews for correctness and coverage.

WHERE IT'S USED

CI pipelines, regression-proofing legacy code, quickly test-covering the class and exception-handling code written earlier today.

REAL-WORLD ANALOGY

"Like handing a junior engineer your class and asking them to draft tests - a useful first draft, but you still review before merging."

Prompting for and Reviewing AI-Generated Tests

KEY TERMINOLOGY

- Test case - a function asserting one expected behavior
- Fixture - reusable setup/teardown for tests (`pytest.fixture`)
- Assertion - a check that fails the test if false (`assert` / `pytest.raises`)
- Coverage - how much of the code's logic paths the tests actually exercise

WRITING A GOOD TEST-GENERATION PROMPT

- Give the assistant the actual source code, not a description of it
- State the framework explicitly: 'using `pytest`', not 'write unit tests'
- Ask explicitly for edge cases: empty input, invalid types, boundary values, raised exceptions

REVIEWING AI-GENERATED TESTS

- Correctness - does each assertion actually match the real expected behavior?
- Coverage - are error paths and boundary conditions tested, not just the happy path?
- Determinism - no reliance on real time, randomness, or network without mocking

DEVELOPER PERSPECTIVE

- Treat AI-generated tests exactly like AI-generated production code - run them, read them, don't just trust a green checkmark

Basic Example

```
example.py

# bank_account.py
class InsufficientFundsError(Exception):
    pass

class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("deposit amount must be positive")
        self._balance += amount
```

WHAT'S HAPPENING

- This is the class from earlier today's OOP and exception-handling work
- Two custom failure modes exist: `ValueError` on bad deposits, `InsufficientFundsError` on bad withdrawals
- A good test suite needs to cover both success paths AND both of these failure paths

Creating Test Cases Using an AI Assistant

WHAT IS IT

Feed the AI assistant your real source module and ask it to generate a PyTest test module covering normal behavior, boundary conditions, and exception paths.

WHY DEVELOPERS NEED IT

It converts a tedious, boilerplate-heavy task into a fast first draft, freeing the developer to focus on reviewing edge cases and correctness instead of typing assert statements.



Creating Test Cases Using an AI Assistant — In Practice

EXAMPLE PROMPT

Using pytest, write a test module for the BankAccount class below. Cover: successful deposit and withdraw, deposit with zero/negative amount raising ValueError, withdraw exceeding balance raising InsufficientFundsError, and the balance property staying accurate after multiple operations. Use pytest.raises for exception cases and clear, descriptive test function names.

before.py

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount): ...
    def withdraw(self, amount): ...
```

after_ai.py

```
# test_bank_account.py
import pytest
from bank_account import BankAccount, InsufficientFundsError

def test_deposit_increases_balance():
    acct = BankAccount("Sam", 100)
    acct.deposit(50)
    assert acct.balance == 150

def test_deposit_rejects_non_positive_amount():
    acct = BankAccount("Sam", 100)
    with pytest.raises(ValueError):
        acct.deposit(0)

def test_withdraw_decreases_balance():
    acct = BankAccount("Sam", 100)
    acct.withdraw(40)
    assert acct.balance == 60

def test_withdraw_raises_when_insufficient_funds():
    acct = BankAccount("Sam", 30)
    with pytest.raises(InsufficientFundsError):
        acct.withdraw(50)
```

Creating Test Cases Using an AI Assistant — Review & Practice

BEFORE YOU TRUST THE OUTPUT — REVIEW FOR:

Correctness

Edge cases

Determinism

Test naming clarity

Coverage of custom exceptions

No hidden dependency on execution order

HANDS-ON ACTIVITY

Paste your own class from an earlier exercise into an AI assistant with a similarly detailed prompt, then run the generated test file with pytest and fix any test that fails for the wrong reason.

Practical Example

```
practical_example.py

# follow-up prompt to the assistant:
# "Add tests for: withdrawing the exact full balance (should succeed),
# depositing a very large amount, and creating an account with a
# negative starting balance."

def test_withdraw_full_balance_succeeds():
    acct = BankAccount("Sam", 100)
    acct.withdraw(100)
    assert acct.balance == 0

def test_deposit_large_amount():
    acct = BankAccount("Sam", 0)
    acct.deposit(1_000_000)
    assert acct.balance == 1_000_000

def test_negative_starting_balance_is_not_validated():
```

WHAT'S HAPPENING

- Iterating means giving the assistant a targeted follow-up prompt, not starting over
- The full-balance-withdraw case is a boundary condition easy to miss by hand
- The negative-starting-balance test documents a real gap: `__init__` doesn't validate balance
- Run with: `pytest -v test_bank_account.py`
- Every round of AI iteration must be re-run, not just visually reviewed

CONSOLE OUTPUT

```
test_bank_account.py::test_deposit_increases_balance PASSED
test_bank_account.py::test_deposit_rejects_negative_amount PASSED
test_bank_account.py::test_withdraw_decreases_balance PASSED
test_bank_account.py::test_withdraw_raises_when_insufficient_funds PASSED
test_bank_account.py::test_withdraw_full_balance_succeeds PASSED
test_bank_account.py::test_deposit_large_amount PASSED
```

Best Practices

PEP 8 & READABILITY

- Name tests test_<behavior>_<condition> so failures are self-explanatory
- One logical assertion focus per test function

MAINTAINABILITY

- Keep generated setup in a shared pytest fixture instead of duplicating it in every test
- Regenerate or update tests whenever the source method's behavior changes

PERFORMANCE

- Keep tests fast and isolated - no real network/DB calls without mocking

Common Mistakes

✗ INCORRECT

```
# accept AI-generated tests  
# without ever running them
```

✗ INCORRECT

```
assert result != None
```

✗ INCORRECT

```
def test_stuff():  
    ...
```

✗ INCORRECT

```
# prompt only asks for happy-path tests
```

✓ CORRECT

```
pytest -v # always execute before trusting or merging
```

✓ CORRECT

```
assert result == expected_value
```

✓ CORRECT

```
def test_withdraw_raises_when_insufficient_funds():  
    ...
```

✓ CORRECT

```
# prompt explicitly asks for boundary  
# and exception/invalid-input cases
```

Common Mistakes — Why It Matters

- Mistake 1: A test that was never run may not even be syntactically valid, or may pass for the wrong reason
- Mistake 2: Overly loose assertions let real regressions slip through silently
- Mistake 3: Vague names make failures meaningless in CI output
- Mistake 4: AI assistants mirror what you ask for - an under-specified prompt yields under-covered tests

Hands-on Exercise

PROBLEM STATEMENT

Take the custom exception-handling function you wrote earlier today (`ship_order` or `transfer_funds`) and prompt an AI assistant to generate a PyTest module for it. Review the output, add one edge case the assistant missed, then run the suite.

INPUT

```
Your ship_order(item, qty, stock) function and its  
InsufficientStockError
```

EXPECTED OUTPUT

```
A test_ship_order.py file; 'pytest -v' output showing all  
tests passing, including at least one edge case you added  
yourself
```

HINTS (no full solution here — give it a try first!)

- Ask the assistant explicitly for a test where `qty` equals `stock[item]` exactly (a boundary case)
- Verify the assistant used `pytest.raises` for the exception path, not a manual `try/except`
- Re-run `pytest` after every edit, don't just read the diff

Day Summary

CONCEPTS LEARNED

- Module vs package, and how the import system resolves names via `sys.path` and the `.pyc` bytecode cache
- Classes as the blueprint/instance model shared with Java/C#/JS, expressed with Python's explicit `self`
- Encapsulation by convention (`_protected`, `__mangled`) rather than enforced access modifiers
- Comprehensions as expression-based iteration - list, dict, set, and lazy generator forms
- Logging levels/handlers as the production replacement for `print()`-based status output
- The `try/except/else/finally` control flow and building a custom exception hierarchy
- Tracebacks as a bottom-up map of the call stack at the moment of failure
- AI-assisted test generation as an iterative, review-then-run workflow, not a one-shot trust exercise

KEY TAKEAWAYS

- Python organizes code into modules/packages the same way you'd expect from any other language, with its own import syntax
- Python OOP is functionally similar to Java/C#/JS OOP; the differences are syntax and convention-based encapsulation, not capability
- Comprehensions are the idiomatic default for simple transforms - readability, not cleverness, decides when to use one
- Logging and structured exception handling are non-negotiable for anything beyond a throwaway script
- Reading a traceback bottom-up is faster than adding another `print()` statement
- AI-generated tests are a fast first draft that still require the same review discipline as AI-generated production code

KEY SYNTAX

- `import x / from x import y / import x as y`
- `sys.path, sys.path.append(...)`
- `class Name: / def __init__(self, ...): / self.attr = value`
- `class Child(Parent): + super().__init__(...)`
- `__str__, __repr__, __eq__` and `@dataclass`
- `[expr for x in it if cond], {k:v for ...}, {expr for ...}, (expr for ...)`
- `logging.getLogger(__name__), logging.basicConfig(level=...)`
- `try / except X as e / else / finally / raise CustomError(...)`
- `breakpoint()`, `pdb` commands `n/s/c/p/l`
- `pytest` test functions and `pytest.raises(...)`

PRACTICAL SKILLS GAINED

- Writing and importing your own Python modules and understanding `sys.path/.pyc` behavior
- Designing classes with attributes, methods, inheritance, and dunder methods, including `@dataclass`
- Rewriting loops as list/dict/set comprehensions and generator expressions appropriately
- Configuring the logging module instead of relying on `print()`
- Implementing `try/except/else/finally` with custom exception classes
- Diagnosing bugs from tracebacks and using `breakpoint()/pdb` interactively
- Prompting an AI assistant to generate, review, and iterate on a PyTest test suite

Inventory Manager: Class, Logging, Exceptions & AI-Generated Tests

Your team needs a small, reusable inventory-tracking module for an internal warehouse tool. It must be importable from other scripts, must never crash silently, and must ship with a reviewed test suite.

REQUIREMENTS

- Create a module `inventory.py` containing an `InventoryItem` class with instance attributes `name`, and a protected `_quantity` (encapsulation convention)
- Add `restock(amount)` and `sell(amount)` instance methods that update `_quantity`
- Define and raise a custom `InsufficientStockError` (subclassing `Exception`) when `sell()` is called for more than is in stock
- Configure a module-level logger (no `print` calls) that logs `INFO` on successful `restock/sell` and `ERROR` when a `sell` fails
- Import and use `InventoryItem` from a separate driver script to demonstrate the import mechanics from today's first topic
- Use an AI coding assistant to generate a `PyTest` test module covering normal operations, the `InsufficientStockError` path, and at least one boundary case (selling exactly the remaining stock)
- Run the generated tests and fix any that fail for the wrong reason before submitting

CONSTRAINTS

- No `print()` anywhere for operational status - use the logging module exclusively
- `InsufficientStockError` must subclass `Exception`, not `BaseException`
- `_quantity` must not be set directly from outside the class in the driver script (respect the convention)
- All generated tests must actually be run with `pytest`, not just visually reviewed

Inventory Manager: Class, Logging, Exceptions & AI-Generated Tests — Input / Output

SAMPLE INPUT

```
item = InventoryItem('widget', 10)
item.restock(5)
item.sell(12)
item.sell(3)  # should raise InsufficientStockError
```

EXPECTED OUTPUT

```
INFO: restocked widget by 5, quantity=15
INFO: sold 12 widget, quantity=3
ERROR: cannot sell 3 widget, only 3 in stock # if this line
is reached after selling exactly 3, it demonstrates the
boundary case instead

pytest -v test_inventory.py
... 5 passed in 0.0Xs
```

HINTS

- `logging.basicConfig(level=logging.INFO)` at the top of the driver script is enough for this exercise
- `raise InsufficientStockError(f"cannot sell {amount} {self.name}, only {self._quantity} in stock")`
- Ask the AI assistant explicitly for the boundary test: selling exactly the remaining quantity should succeed, one more than that should raise

OPTIONAL CHALLENGE

Extend the module with a `@dataclass`-based `PriceTag(amount, currency)` attached to each `InventoryItem`, and add a generator expression that yields the names of items below a low-stock threshold.